

# MERN Full Stack Final Assessment (FA)

## SET E — Learning & Course Management System API Response & Frontend Evaluator Contract

### 1. Purpose of This Document

This document defines the:

- expected API response formats
- status codes
- pagination structures
- authentication response structures
- analytics response formats
- frontend evaluator conventions
- Playwright compatibility requirements

for the:

#### Learning & Course Management System — Set E

Students must strictly follow:

- route names
- response structures
- status codes
- data-testid conventions
- window.appState structure

mentioned in this document.

Failure to follow these conventions may affect automated evaluation.

### 2. Common Backend Response Standards

#### Success Response

##### Status Code

200 / 201

##### Expected Structure

```
{
  "success": true,
  "message": "Operation successful",
  "data": {
    "_id": "string",
    "createdAt": "ISO_DATE",
    "updatedAt": "ISO_DATE"
  }
}
```

## Error Response

### Status Codes

| Scenario         | Status |
|------------------|--------|
| Validation Error | 400    |
| Unauthorized     | 401    |
| Forbidden        | 403    |
| Not Found        | 404    |
| Server Error     | 500    |

### Expected Structure

```
{
  "success": false,
  "message": "Error message"
}
```

## Pagination Response Format

### Expected Structure

```
{
  "success": true,
  "message": "Data fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string"
    }
  ]
}
```

## 3. MongoDB & Authentication Routes

### POST /sync

#### Status Code

200

#### Auth Required

Yes

**Allowed Roles**

admin

coordinator

**Expected Response**

```
{
  "success": true,
  "message": "Dataset synchronized successfully",
  "data": {
    "totalFetched": 150,
    "inserted": 120,
    "duplicates": 12,
    "rejected": 18
  }
}
```

Values are dynamic based on dataset.

**GET /health****Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Database connected successfully",
  "data": {
    "database": "connected",
    "documentCount": 120
  }
}
```

**POST /auth/register****Status Code**

201

**Expected Response**

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "_id": "string",
    "userId": "USR101",
    "name": "Dinesh",
    "email": "dinesh@test.com",
  }
}
```

```
{
  "role": "student",
  "createdAt": "ISO_DATE"
}
```

## POST /auth/login

### Status Code

200

### Expected Response

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "_id": "string",
    "userId": "USR101",
    "name": "Dinesh",
    "email": "dinesh@test.com",
    "role": "instructor"
  }
}
```

## GET /auth/me

### Status Code

200

### Auth Required

Yes

### Expected Response

```
{
  "success": true,
  "message": "Authenticated user fetched successfully",
  "data": {
    "_id": "string",
    "userId": "USR101",
    "name": "Dinesh",
    "email": "dinesh@test.com",
    "role": "student"
  }
}
```

## 4. User APIs

### GET /users

#### Status Code

200

#### Pagination Supported

Yes

#### Expected Response

```
{
  "success": true,
  "message": "Users fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string",
      "userId": "USR101",
      "name": "Dinesh",
      "email": "dinesh@test.com",
      "role": "student",
      "status": "active"
    }
  ]
}
```

### GET /users/:id

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "User fetched successfully",
  "data": {
    "_id": "string",
    "userId": "USR101",
    "name": "Dinesh",
    "email": "dinesh@test.com",
    "role": "student",
    "department": "CSE",
    "status": "active"
  }
}
```

```
}
```

## 5. Course APIs

### POST /courses

#### Status Code

201

#### Expected Response

```
{
  "success": true,
  "message": "Course created successfully",
  "data": {
    "_id": "string",
    "courseId": "COURSE101",
    "title": "React Basics",
    "category": "Web",
    "status": "active",
    "createdAt": "ISO_DATE"
  }
}
```

### GET /courses

#### Status Code

200

#### Pagination Supported

Yes

#### Expected Response

```
{
  "success": true,
  "message": "Courses fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string",
      "courseId": "COURSE101",
      "title": "React Basics",
      "category": "Web",
      "status": "active"
    }
  ]
}
```

```
]
}
```

**GET /courses/:id****Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Course fetched successfully",
  "data": {
    "_id": "string",
    "courseId": "COURSE101",
    "title": "React Basics",
    "category": "Web",
    "duration": 40,
    "level": "beginner",
    "instructor": {},
    "status": "active"
  }
}
```

**PATCH /courses/:id****Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Course updated successfully",
  "data": {
    "_id": "string",
    "courseId": "COURSE101",
    "status": "ongoing",
    "updatedAt": "ISO_DATE"
  }
}
```

**DELETE /courses/:id****Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Course deleted successfully"
}
```

## 6. Enrollment APIs

### POST /enrollments

#### Status Code

201

#### Expected Response

```
{
  "success": true,
  "message": "Enrollment created successfully",
  "data": {
    "_id": "string",
    "enrollmentId": "ENR101",
    "student": {},
    "course": {},
    "progress": 0,
    "completionStatus": "ongoing",
    "createdAt": "ISO_DATE"
  }
}
```

### GET /enrollments

#### Status Code

200

#### Pagination Supported

Yes

#### Expected Response

```
{
  "success": true,
  "message": "Enrollments fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string",
      "enrollmentId": "ENR101",

```



```
    "completionStatus": "ongoing",
    "progress": 65
  }
]
```

### GET /enrollments/:id

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "Enrollment fetched successfully",
  "data": {
    "_id": "string",
    "enrollmentId": "ENR101",
    "student": {},
    "course": {},
    "progress": 65,
    "completionStatus": "ongoing",
    "enrolledAt": "ISO_DATE"
  }
}
```

### PATCH /enrollments/:id

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "Enrollment updated successfully",
  "data": {
    "_id": "string",
    "enrollmentId": "ENR101",
    "progress": 80,
    "updatedAt": "ISO_DATE"
  }
}
```

### DELETE /enrollments/:id

#### Status Code

200

**Expected Response**

```
{
  "success": true,
  "message": "Enrollment deleted successfully"
}
```

## 7. Assessment APIs

**POST /assessments****Status Code**

201

**Expected Response**

```
{
  "success": true,
  "message": "Assessment created successfully",
  "data": {
    "_id": "string",
    "assessmentId": "ASM101",
    "title": "React Quiz",
    "type": "quiz",
    "marks": 85,
    "grade": "A",
    "createdAt": "ISO_DATE"
  }
}
```

**GET /assessments****Status Code**

200

**Pagination Supported**

Yes

**Expected Response**

```
{
  "success": true,
  "message": "Assessments fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string",
```

```
{
  "assessmentId": "ASM101",
  "title": "React Quiz",
  "marks": 85,
  "grade": "A"
}
```

### GET /assessments/:id

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "Assessment fetched successfully",
  "data": {
    "_id": "string",
    "assessmentId": "ASM101",
    "title": "React Quiz",
    "type": "quiz",
    "student": {},
    "course": {},
    "marks": 85,
    "grade": "A",
    "submittedAt": "ISO_DATE"
  }
}
```

### PATCH /assessments/:id

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "Assessment updated successfully",
  "data": {
    "_id": "string",
    "assessmentId": "ASM101",
    "marks": 90,
    "grade": "A+",
    "updatedAt": "ISO_DATE"
  }
}
```

```
}
```

**DELETE /assessments/:id****Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Assessment deleted successfully"
}
```

**8. Learning Workflow APIs****PATCH /courses/:id/assign****Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Instructor assigned successfully",
  "data": {
    "courseId": "COURSE101",
    "instructor": {
      "_id": "string",
      "name": "Rahul"
    }
  }
}
```

**PATCH /courses/:id/status****Allowed Status**

active

ongoing

completed

archived

**Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Course status updated successfully",
}
```

```
"data": {
  "courseId": "COURSE101",
  "status": "completed",
  "updatedAt": "ISO_DATE"
}
```

### Workflow Validation Rules

- archived courses cannot move back to active
- completed courses cannot accept enrollments
- students cannot modify course status
- instructors cannot archive courses directly

## 9. Filtering, Search & Pagination APIs

**GET /courses?category=Web**

**GET /courses?level=advanced**

**GET /courses?status=active**

**Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Courses filtered successfully",
  "data": [
    {
      "_id": "string",
      "courseId": "COURSE101",
      "category": "Web"
    }
  ]
}
```

**GET /enrollments?completionStatus=completed**

**GET /enrollments?progressMin=70**

**Status Code**

200

**Expected Response**

```
{
  "success": true,
  "message": "Enrollments filtered successfully",
```

```
"data": [  
  {  
    "_id": "string",  
    "enrollmentId": "ENR101",  
    "completionStatus": "completed"  
  }  
]  
}
```

**GET /courses?page=1&limit=10**

**GET /courses?search=react**

**GET /courses?category=Web&status=active&page=2&limit=5**

**Status Code**

200

**Expected Pagination Response**

```
{  
  "success": true,  
  "message": "Courses fetched successfully",  
  "page": 2,  
  "limit": 5,  
  "total": 50,  
  "totalPages": 10,  
  "data": [  
    {  
      "_id": "string",  
      "courseId": "COURSE101",  
      "title": "React Basics"  
    }  
  ]  
}
```

## 10. Analytics APIs

**GET /analytics/learning**

**Status Code**

200

**Expected Response**

```
{  
  "success": true,  
  "message": "Learning analytics fetched successfully",  
  "data": {  
    "totalStudents": 200,  
  }  
}
```

```
{
  "activeCourses": 25,
  "completedEnrollments": 120,
  "averageCompletionPercentage": 74
}
```

Values are dynamic.

### GET /analytics/courses

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "Course analytics fetched successfully",
  "data": [
    {
      "course": "React Basics",
      "enrollmentCount": 45,
      "highestCompletionRate": 92,
      "activeInstructorCount": 8,
      "archivedCourses": 3
    }
  ]
}
```

Values are dynamic.

### GET /analytics/assessments

#### Status Code

200

#### Expected Response

```
{
  "success": true,
  "message": "Assessment analytics fetched successfully",
  "data": {
    "averageMarks": 78,
    "gradeDistribution": {
      "A": 20,
      "B": 35,
      "C": 18
    },
    "topPerformingStudents": 12,
    "failedAssessmentsCount": 5
  }
}
```



```
}  
}
```

Values are dynamic.

## 11. Frontend Evaluator Contract

### Mandatory React Router Routes

| Route        | Purpose               |
|--------------|-----------------------|
| /login       | Authentication        |
| /dashboard   | Analytics dashboard   |
| /courses     | Course management     |
| /enrollments | Enrollment management |
| /assessments | Assessment management |
| /students    | Student management    |
| /profile     | Current user profile  |

### Mandatory window.appState Structure

Students must expose:

window.appState

### Expected Structure

```
window.appState = {  
  authUser,  
  token,  
  users,  
  courses,  
  enrollments,  
  assessments,  
  filters,  
  analytics  
}
```

### Runtime State Validation

The evaluator may validate state updates dynamically after:

- POST operations
- PUT operations
- PATCH operations
- DELETE operations





## SRET

- login/logout actions
- filtering/search actions
- pagination actions